

Introduction to CAN Transmission with STM32 HAL

Austin Yang ∈ Illini RoboMaster

[austiny2@illinois.edu, <https://www.illinirobomaster.com>]

September 27, 2025

CONTENTS

1	INTRODUCTION	3
1(A)	PLAN	3
1(B)	BASIC KNOWLEDGE	3
1(B).1	CAN PROTOCOL BASICS	3
1(B).2	CAN FRAME STRUCTURE	3
2	UNDERSTANDING RAW CAN HARDWARE SETUP	8
2(A)	CAN BUS HARDWARE	8
2(A).1	CAN BUS TERMINATION	8
2(A).2	TERMINOLOGY	8
2(A).3	BAUD RATE	8
2(B)	CAN MESSAGE ARBITRATION AND ACKNOWLEDGEMENT	9
2(B).1	STANDARD VS EXTENDED IDS	10
2(B).2	DATA VS REMOTE FRAMES	10
3	UNDERSTANDING THE CAN TUTORIAL PROGRAM	11
3(A)	INCLUDE STATEMENTS AND COPYRIGHT	11
3(B)	RAW CAN FRAME TRANSMISSION FUNCTION	12
3(B).1	PARAMETER VALIDATION	12
3(B).2	PRINTING?	13
3(B).3	CAN TRANSMISSION HEADER CONFIGURATION	14
3(B).4	SENDING THE CAN MESSAGE	14
3(B).5	WAITING FOR TRANSMISSION COMPLETION	15
3(B).6	TRANSMISSION CONFIRMATION	16
3(C)	CAN INITIALIZATION FUNCTION	16
3(C).1	HARDWARE INITIALIZATION	16
3(C).2	RTOS INIT FUNCTION	17

3(D)	DEFAULT TASK IMPLEMENTATION	17
3(D).1	MAIN TASK LOOP	17
4	CONCEPTS COVERED	19
4(A)	CAN PROTOCOL	19
4(B)	STM32 HAL CAN	19
4(C)	EMBEDDED PROGRAMMING	19
5	CONCLUSION	20

1 INTRODUCTION

1(A) PLAN

Last time, we went through the *object layer* implementation for CAN on a Linux board. This time, we will go through the *transfer layer* specification and a toy implementation.

This tutorial will go through `can_raw_transmit.cc` line-by-line as an introduction to CAN (Controller Area Network) communication at a low level. We will cover concepts in:

- CAN protocol fundamentals
- Hardware Abstraction Layer (HAL) for CAN communication
- Raw CAN frame transmission
- STM32 microcontroller CAN implementation

You can find the files covered/mentioned at

https://github.com/illini-robomaster/iRM_Embedded_2026/blob/embedded_tutorial/examples/motor/can_raw_transmit.cc

https://github.com/illini-robomaster/iRM_Embedded_2026/blob/embedded_tutorial/shared/bsp/bsp_can.h

https://github.com/illini-robomaster/iRM_Embedded_2026/blob/embedded_tutorial/shared/bsp/bsp_can.cc

1(B) BASIC KNOWLEDGE

1(b).1 CAN Protocol Basics

CAN is a communication protocol (“bus”) that allows devices to talk to each other in a distributed manner. It was first used on cars, but now is a standard protocol in a variety of fields, including robotics.

It is a mailbox-based system, which means that messages are broadcast to all devices connected to the network. Each device has an ID, and only care about the messages that are marked with an ID they care about (typically, their own).

While we can only specify a single ID in our CAN frame (we will see this very explicitly in [subsubsection 1\(b\).2](#)), we can use some tricks to talk to multiple devices with a single packet. We will see an example of this later in our own motor communication code.

1(b).2 CAN Frame Structure

The following information can all be found on Wikipedia: https://en.wikipedia.org/wiki/CAN_bus. I have reworded it and added some comments. I have also changed the color coding scheme. I have colored the entries that we care about, mostly.

This is the structure of the standard CAN packet:

Field	Bits	Purpose
SOF	1	“ <u>S</u> tart <u>o</u> f <u>F</u> rame”. You can guess what this does.
Identifier	11	Contains a standard 11-bit CAN ID.
RTR	1	“ <u>R</u> emote <u>T</u> ransmission <u>R</u> equest”. This specifies the type of frame the message uses.
IDE	1	“ <u>I</u> dentifier <u>E</u> xtension”. This specifies whether we use a standard CAN frame (0) or extended CAN frame (1).
r0	1	Reserved.
DLC	4	“ <u>D</u> ata <u>L</u> ength <u>C</u> ode”. Specifies how many bytes are in the data.
Data Field	0-64	Contains 0 to 8 bytes of the actual data.
CRC	15	“ <u>C</u> yclic <u>R</u> edundancy <u>C</u> heck”. This is used for detection and correction of errors. This will be explained in the future.
CRC Delimiter	1	Marks end of CRC.
ACK Slot	1	“ <u>A</u> cknowledgement slot”. Receivers set this bit to tell the transmitter they received a message.
ACK Delimiter	1	Marks end of ACK.
EOF	7	“ <u>E</u> nd <u>o</u> f <u>F</u> rame”. You can guess what this does.
IFS	3	“ <u>I</u> nter-frame spacing”. This isn’t a part of the frame. This is the minimum distance between two CAN frames.

Table 1: Explanation of each field in a standard CAN 2.0A frame

This is the structure of the extended CAN packet:

Field	Bits	Purpose
SOF	1	“ <u>S</u> tart <u>o</u> f <u>F</u> rame”. You can guess what this does.
Identifier A	11	The first 11 bits of the CAN ID.
SRR	1	“ <u>S</u> ubstitute <u>R</u> emote <u>R</u> equest”. This is set to recessive. I will explain why in subsection 2(b) .
IDE	1	“ <u>I</u> dentifier <u>E</u> xtension Bit”. This declares we are using an extended frame. This must be set to recessive (1).
Identifier B	18	Contains the 18-bit ID extension.
RTR	1	“ <u>R</u> emote <u>T</u> ransmission <u>R</u> equest”. This specifies the type of frame the message uses.
r1	1	Reserved.
r0	1	Reserved.
DLC	4	“ <u>D</u> ata <u>L</u> ength <u>C</u> ode”. Specifies how many bytes are in the data.
Data Field	0-64	Contains 0 to 8 bytes of the actual data.
CRC	15	“ <u>C</u> yclic <u>R</u> edundancy <u>C</u> heck”. This is used for detection and correction of errors. This will be explained in the future.
CRC Delimiter	1	Marks end of CRC.
ACK Slot	1	“ <u>A</u> cknowledgement slot”. Receivers set this bit to tell the transmitter they received a message.
ACK Delimiter	1	Marks end of ACK.
EOF	7	“ <u>E</u> nd <u>o</u> f <u>F</u> rame”. You can guess what this does.
IFS	3	“ <u>I</u> nter-frame <u>s</u> pacing”. This isn’t a part of the frame. This is the minimum distance between two CAN frames.

Table 2: Explanation of each field in an extended CAN 2.0B frame

Here is the `CAN_TxHeaderTypeDef` structure from `stm32f4xx_hal_can.h`. We can see some of the fields we are allowed to set in the packet header.

```

148  /**
149   * @brief CAN Tx message header structure definition
150   */
151  typedef struct
152  {
153      /*!< Specifies the standard identifier.
154       This parameter must be a number between Min_Data = 0 and Max_Data = 0x7FF. */
155      uint32_t StdId;
156
157      /*!< Specifies the extended identifier.
158       This parameter must be a number between Min_Data = 0 and Max_Data = 0x1FFFFFFF. */
159      uint32_t ExtId;
160
161      /*!< Specifies the type of identifier for the message that will be transmitted.
162       This parameter can be a value of @ref CAN_identifier_type */
163      uint32_t IDE;
164
165      /*!< Specifies the type of frame for the message that will be transmitted.
166       This parameter can be a value of @ref CAN_remote_transmission_request */
167      uint32_t RTR;
168
169      /*!< Specifies the length of the frame that will be transmitted.
170       This parameter must be a number between Min_Data = 0 and Max_Data = 8. */
171      uint32_t DLC;
172
173      /*!< Specifies whether the timestamp counter value captured on start
174       of frame transmission, is sent in DATA6 and DATA7 replacing pData[6] and pData[7].
175       @note: Time Triggered Communication Mode must be enabled.
176       @note: DLC must be programmed as 8 bytes, in order these 2 bytes are sent.
177       This parameter can be set to ENABLE or DISABLE. */
178      FunctionalState TransmitGlobalTime;
179
180  } CAN_TxHeaderTypeDef;

```

Note I changed the spacing to make it fit on the page. Only the first line number is correct.

TIP | When we say “set a bit” or “mark a bit” this usually means that we change the bit to a logical 1.

REMARK | The prefix `0x` means that we are representing a hexadecimal (base-16) number. This syntax is common in programming. Here are some others in C++:

- `0b`: Binary.
- `0o`: Octal.

Ending the number with a `u` (i.e. `114514u`) means that the number is *unsigned*. Others:

- **L**: Long.
- **LL**: Long long.

If you are unfamiliar with numerical representations, I recomend you to read the following Wikipedia articles:

https://en.wikipedia.org/wiki/Signed_number_representations

https://en.wikipedia.org/wiki/Two%27s_complement

https://en.wikipedia.org/wiki/IEEE_754

2 UNDERSTANDING RAW CAN HARDWARE SETUP

2(A) CAN BUS HARDWARE

The CAN bus is physically implemented using two wires, CAN high and CAN low, which carry the signals.

REMARK | We usually have a separate pair of wires for power (VCC) and ground (GND), so our CAN typically takes 4 wires.

TIP | A CAN device can be any device that uses the CAN protocol. We connect CAN devices with CAN bus wires in a web, in any way we want.

2(a).1 CAN Bus Termination

The CAN bus must be terminated with 120Ω resistors at both ends to prevent reflections that can cause data corruption. You will learn this in ECE 329 if you choose to take it. Without this the bus will be unreliable.

REMARK | The *characteristic impedance* (represented by η_0) of a standard CAN bus cable is around 120Ω . This is what we are trying to match.

2(a).2 Terminology

- **CAN-H:** High signal line (around the voltage of VCC)
- **CAN-L:** Low signal line (around the voltage of GND)
- **Twisted Pair:** The lines are twisted together to improve electromagnetic performance.
- **Differential Signaling:** Use the voltage difference between CAN-H and CAN-L, not CAN-H and GND.
- **Recessive:** A low voltage bit. This represents a logical 1.
- **Dominant:** A high voltage bit. This represents a logical 0.

Exercise: Why might we want to use high voltage as a logical 0? (There are many benefits.)

2(a).3 Baud Rate

The *baud rate* is the frequency at which the protocol operates. It is the number of bits transmitted per second. We use a baud rate of 115200.

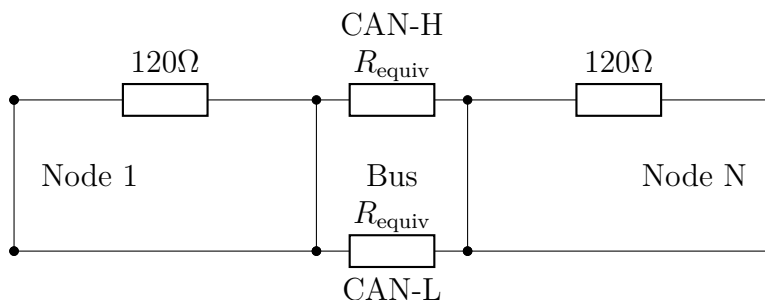


Figure 1: CAN Bus Terminals

2(B) CAN MESSAGE ARBITRATION AND ACKNOWLEDGEMENT

We need to determine the priority of transmission when multiple devices are in a network. We take advantage of the fact that every message begins with an ID.

Every device can see the current state of the bus (high or low voltage). They are also synchronized. Note that when a dominant bit and recessive bit are sent at the same time, the dominant bit (high voltage, logical 0) wins out.

Therefore, when transmitting, all devices that send attempt transmitting the packet, starting with the ID. They also all observe the bus. When they try to send a recessive bit (low voltage, logical 1) but notice a dominant bit (high voltage, logical 0) is on the bus, they stop and wait before retransmitting. The effect is that lower indexed IDs have priority. It also turns out that standard CAN frames *naturally* have priority over extended CAN frames.

Exercise: try to reason this out by yourself. Here is a handy diagram:

Bit(s)	1	2-12	13	14	...	...
Standard	SOF	ID	RTR	IDE	...	EOF
Extended	SOF	ID A	SRR	IDE	...	EOF

Hint: In a standard frame, RTR can be recessive or dominant. The same bit in the extended frame is always _____. The 14th bit in each frame is both the IDE, so the _____ frame always wins out by sending a _____ bit, causing the other one to stop and wait.

REMARK | This comparison happens after the first 11 bits of both IDs are compared. In addition, the RTR bit may win over the SRR bit, but the SRR bit can never win over the RTR bit.

Read https://en.wikipedia.org/wiki/CAN_bus#Data_transmission_and_arbitration for additional information.

Notice that there is an ACK bit set aside for acknowledgement. This is used by the receiver to tell the transmitter it has successfully gotten an uncorrupted message. Notice its location in the packet:

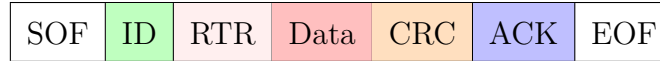


Table 3: Simplified CAN Frame Diagram

It is *after* receiving both the data and CRC.

When the receiver verifies the data received is correct using the CRC, it sends a dominant signal into the wire for a length of time corresponding to one bit. This tells the transmitter the receiver has seen the correct message.

REMARK | We only ACK if the CRC checksum shows the data is uncorrupted. If not, we *want* a retransmit.

2(b).1 Standard vs Extended IDs

- **Standard ID:** 11-bit identifier (0x000 to 0x7FF), used in CAN 2.0A (standard) and CAN 2.0B (extended).
- **Extended ID:** 29-bit identifier (0x00000000 to 0x1FFFFFFF), used in CAN 2.0B.
- Same arbitration mechanism!

We will not be using the extended bits.

2(b).2 Data vs Remote Frames

- **Data Frame:** Contains actual data payload (RTR bit is set to 0).
- **Remote Frame:** Often used to request data from another node (RTR bit is set to 1, no data field).

3 UNDERSTANDING THE CAN TUTORIAL PROGRAM

Let's examine the implementation file `bsp_can_raw_transmit.cc` to understand how raw CAN frames are transmitted.

3(A) INCLUDE STATEMENTS AND COPYRIGHT

```
1  /*****
2  *
3  *  Copyright (C) 2025 RoboMaster.
4  *  Illini RoboMaster @ University of Illinois at Urbana-Champaign
5  *
6  *  This program is free software: you can redistribute it and/or modify
7  *  it under the terms of the GNU General Public License as published by
8  *  the Free Software Foundation, either version 3 of the License, or
9  *  (at your option) any later version.
10 *
11 *  This program is distributed in the hope that it will be useful,
12 *  but WITHOUT ANY WARRANTY; without even the implied warranty of
13 *  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
14 *  GNU General Public License for more details.
15 *
16 *  You should have received a copy of the GNU General Public License
17 *  along with this program. If not, see <http://www.gnu.org/licenses/>.
18 *
19 *****/
20
21 #include "bsp_print.h"
22 #include "cmsis_os.h"
23 #include "main.h"
24 #include "can.h"
```

The file starts with a copyright notice. You should be familiar with this from the previous tutorial.

The includes bring in the necessary libraries for CAN communication on STM32:

- `bsp_print.h`: Board support package for printing/debugging.
- `cmsis_os.h`: CMSIS Real-Time Operating System API.
- `main.h`: Board `#define`s and basic functionality is in there. We simply write i.e. the main loop in this file.
- `can.h`: CAN.

REMARK | BSP stands for board support package.

3(B) RAW CAN FRAME TRANSMISSION FUNCTION

```
26 /**
27  * @brief Send a raw CAN frame using HAL directly
28  * @param can_handle Pointer to CAN handle (e.g., &hcan1)
29  * @param id CAN ID (11-bit standard ID)
30  * @param data Data buffer to send (up to 8 bytes)
31  * @param length Data length (0-8 bytes)
32  * @return HAL status (HAL_OK if successful)
33  */
34 HAL_StatusTypeDef SendRawCANFrame(CAN_HandleTypeDef* can_handle,
35                                   uint32_t id, uint8_t* data, uint8_t length) {
```

This function definition and the docstring above shows the signature for sending a raw CAN frame using the STM32 HAL (Hardware Abstraction Layer). (Note I split the last lines into two lines in order to fit it.)

After reading the previous tutorial, try to identify what each part means by yourself. Exercise: We never included `<stdint.h>`. Why can we still use `uint32_t` and `uint8_t`?

TIP | Headers included by other file A are available when you include file A.

REMARK | This is what `#pragma once` and header guards are for.

3(b).1 Parameter Validation

```
35 // Validate parameters
36 if (length > 8) {
37     print("Error: CAN data length exceeds 8 bytes\r\n");
38     return HAL_ERROR;
39 }
40
41 if (id > 0x7FF) {
42     print("Error: Standard CAN ID exceeds 11-bit range (0x7FF)\r\n");
43     return HAL_ERROR;
44 }
```

The function first validates the input parameters to ensure they comply with CAN protocol specifications:

- CAN data length cannot exceed 8 bytes (per CAN 2.0 specification)
- Standard CAN ID must fit in 11 bits (0x000 to 0x7FF)

This validation prevents protocol violations and potential communication failures.

TIP | We end each `print` with `\r\n`. `\r` and `\n` are both flavors of “new line”. We use both to ensure compatibility with all computers.

REMARK | Most computers and programs are fine with just `\n`. However, there are still many programs that use `\r` or both.

3(b).2 Printing?

Have you noticed a problem? We just used `print`. But we are on an STM32, which is an embedded system usually with no screens attached! What are we even printing to? Where does `print` even come from?

REMARK | `print` comes from `bsp_print.h`.

Notice when we introduced `bsp_print.h`, we mentioned it is for printing *and debugging*. We are actually outputting bytes into another serial port using a standard protocol called *UART*, so that we can plug in our computer at the other end to debug.

Indeed, let us look at the actual definition of `print` in `shared/bsp/bsp_print.cc`:

```
49 int32_t print(const char* format, ...) {
50     #ifdef NDEBUG
51         UNUSED(format);
52         UNUSED(print_buffer);
53         return 0;
54     #else // == #ifdef DEBUG
55         va_list args;
56         int length;
57
58         va_start(args, format);
59         length = vsnprintf(print_buffer, MAX_PRINT_LEN, format, args);
60         va_end(args);
61
62         if (print_uart)
63             return print_uart->Write((uint8_t*)print_buffer, length);
64         else if (print_usb)
65             return print_usb->Write((uint8_t*)print_buffer, length);
66         else
67             return 0;
68     #endif // #ifdef NDEBUG
69 }
```

It is writing to UART! We will cover UART in a future tutorial.

REMARK | We usually use `print_use_uart` to set this up. Here is the declaration located in `bsp_print.h`.

```
27 /**
28  * @brief use a uart port for debug print
29  *
30  * @param huart HAL uart handle
31  */
32 void print_use_uart(UART_HandleTypeDef* huart);
```

3(b).3 CAN Transmission Header Configuration

(I de-indented the comment on line 49 up so you can see the whole thing.)

REMARK | Notice I did not have to do any of this in the previous tutorial! This is why writing 80-character lines is the standard. Take heed! (CC: THE RSO PRESIDENT)

```
46 // Configure CAN transmission header
47 CAN_TxHeaderTypeDef tx_header;
48 tx_header.StdId = id; // Standard ID (11-bit)
49 tx_header.ExtId = 0x0; // Extended ID (not used for standard frames)
50 tx_header.IDE = CAN_ID_STD; // Use standard ID format
51 tx_header.RTR = CAN_RTR_DATA; // Data frame (not remote frame)
52 tx_header.DLC = length; // Data length code
53 tx_header.TransmitGlobalTime = DISABLE; // Don't include timestamp
```

Here we configure the CAN transmission header structure with the following parameters:

- **StdId**: The 11-bit standard CAN identifier (0x000 to 0x7FF).
- **ExtId**: Extended identifier (ignored as we are using standard frames).
- **IDE**: Identifier extension bit. Set to **CAN_ID_STD** (alias for 0x00000000U - remember what this means?) for standard format (CAN 2.0A).
- **RTR**: Remote transmission request. Set to **CAN_RTR_DATA** (also 0x00000000U) for data frames.
- **DLC**: Data length code. This is the number of data bytes to send (0-8).
- **TransmitGlobalTime**: Whether to include global time stamp in transmission (used for synchronization, disabled).

3(b).4 Sending the CAN Message

(Note I had to split line 56 to fit it.)

```
55 uint32_t tx_mailbox;
56 HAL_StatusTypeDef status = HAL_CAN_AddTxMessage(can_handle, &tx_header, data,
57                                                &tx_mailbox);
58
59 if (status != HAL_OK) {
60     print("Error: Failed to add CAN message to TX mailbox, status: %d\r\n", status);
61     return status;
62 }
```

The **HAL_CAN_AddTxMessage** function adds the message to one of the CAN hardware mailboxes for transmission. Let's look at its implementation in `stm32f4xx_hal_can.c` (the header doesn't contain the docstring I want to show you) (PPS I split line 1250 to fit it):

```

1238 /**
1239  * @brief Add a message to the first free Tx mailbox and activate the
1240  *        corresponding transmission request.
1241  * @param hcan pointer to a CAN_HandleTypeDef structure that contains
1242  *        the configuration information for the specified CAN.
1243  * @param pHeader pointer to a CAN_TxHeaderTypeDef structure.
1244  * @param aData array containing the payload of the Tx frame.
1245  * @param pTxMailbox pointer to a variable where the function will return
1246  *        the TxMailbox used to store the Tx message.
1247  *        This parameter can be a value of @arg CAN_Tx_Mailboxes.
1248  * @retval HAL status
1249  */
1250 HAL_StatusTypeDef HAL_CAN_AddTxMessage(CAN_HandleTypeDef *hcan,
1251     CAN_TxHeaderTypeDef *pHeader, uint8_t aData[], uint32_t *pTxMailbox)

```

Then we know:

- `can_handle`: The CAN device handle.
- `tx_header`: Reference to the configured transmission header.
- `data`: The data buffer to send.
- `tx_mailbox`: Output parameter that receives which mailbox is to be used.

This function returns a status indicating success or failure, which we check and respond to appropriately.

REMARK | *Handle* is jargon for some abstract object or token used to represent a resource, in this case a CAN device. You will encounter this term very often in embedded programming.

Exercise: What played the role of `tx_mailbox` in our previous tutorial about Linux CAN?
Hint: was it `ftx` or `frx`?

3(b).5 Waiting for Transmission Completion

```

55 // Wait for transmission to complete
56 while (HAL_CAN_IsTxMessagePending(can_handle, tx_mailbox)) {
57     osDelay(1); // Small delay to prevent busy waiting
58 }

```

After queuing the message for transmission, we check if it's still pending in the mailbox:

- `HAL_CAN_IsTxMessagePending()`: Returns whether transmission is still in progress.
- `osDelay(1)`: A small delay (measured in “ticks”).

This ensures the message is fully transmitted before the function returns.

3(b).6 Transmission Confirmation

```
59 print("CAN Frame sent successfully - ID: 0x%03X, Length: %d, Mailbox: %d\r\n",
60       (unsigned int)id, length, (unsigned int)tx_mailbox);
61
62 return HAL_OK;
```

Finally, we print a confirmation message showing the transmitted ID, data length, and mailbox used, then return `HAL_OK` to indicate success.

TIP | Notice `%03X` and `%d`. These are *format specifiers*. Read <https://cplusplus.com/reference/cstdio/printf> for more.

3(c) CAN INITIALIZATION FUNCTION

```
74 /**
75  * @brief Initialize CAN and start the peripheral
76  */
77 void InitializeRawCAN() {
78     // Initialize CAN1
79     MX_CAN1_Init();
80
81     // Start CAN peripheral
82     if (HAL_CAN_Start(&hcan1) != HAL_OK) {
83         print("Error: Failed to start CAN1\r\n");
84         Error_Handler();
85     }
86
87     print("CAN1 initialized and started successfully\r\n");
88     print("Ready to send raw CAN frames on hcan1\r\n");
89 }
```

3(c).1 Hardware Initialization

The initialization process involves two steps:

- `MX_CAN1_Init`: Generated by STM32CubeMX to configure the CAN1 peripheral settings.
- `HAL_CAN_Start`: Starts the CAN hardware and enables communication

If the start operation fails, an error is printed and the error handler is called to manage the fault condition.

REMARK | *Peripheral* is jargon for the actual hardware module in the microcontroller that implements the protocol, i.e. a peripheral device. Again, this is a common term in embedded programming.

3(c).2 RTOS Init Function

```
96 void RM_RTOS_Init() {  
97     print_use_uart(&huart1);  
98     InitializeRawCAN();  
99 }
```

This is the init function defined in `main.h`. It is run before the main loop and is used for setup and initialization. Notice the functions we are calling inside of it.

3(D) DEFAULT TASK IMPLEMENTATION

```
83 void RM_RTOS_Default_Task(const void* args) {  
84     UNUSED(args);  
85  
86     print("Raw CAN Transmission Example\r\n");  
87     print("Sending various CAN frames every 2 seconds...\r\n");  
88  
89     uint32_t counter = 0;  
90  
91     while (true) {  
92         // Example 1: Send motor command (similar to DJI motor protocol)  
93         uint8_t motor_data[8] = {0x03, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};  
94         SendRawCANFrame(&hcan1, 0x200, motor_data, 8);  
95  
96  
97         osDelay(2);  
98  
99  
100        print("=== Cycle %lu completed ===\r\n", counter);  
101    }  
102 }
```

TIP | Notice the double newlines. This is bad formatting, don't do it.

3(d).1 Main Task Loop

The main task implements an infinite loop (the “main loop”) that:

- Sends a CAN frame with ID `0x200` containing motor control data.
- Waits between transmissions.
- Prints a counter to track completed cycles.

REMARK | We can concurrently control up to 4 motors using a single CAN packet! All four motors listen for the ID `0x200` and decide what parts of the data to use based on

their TX ID. We talked about this last time in person, but will write about it in the next tutorial (on motors). In the meantime, read the “RoboMaster C620 Brushless DC Motor Speed Controller V1.01” datasheet we have sent out to you for more information.

4 CONCEPTS COVERED

We went over the following:

4(A) CAN PROTOCOL

- CAN frame structure and transmission
- Bus termination and wiring requirements
- Message arbitration based on CAN ID
- Acknowledgement
- Standard (2.0A) vs extended (2.0B) frame formats

4(B) STM32 HAL CAN

- CAN hardware abstraction layer usage
- CAN peripheral initialization
- Transaction header configuration
- Mailbox-based message transmission

4(C) EMBEDDED PROGRAMMING

- RTOS functions

5 CONCLUSION

This tutorial introduced raw CAN transmission on an STM32 embedded board. We covered the basics of CAN protocol, hardware setup, and a very simple usage of it to send a CAN packet (if you were here in person, we made a motor spin using this!). Next time I will go over the basics of motors and maybe CRC.

Same as last time: if there are any concepts I have failed to cover, or you do not completely understand, search it up and ask an LLM.